



DON'T DROP THE **BATON**: LONG-HORIZON ROBOT MANIPULATION VIA AGENTIC SUBTASK EXPLORATION AND TRANSITION-AWARE MEMORY

Bingxin Xu¹ Yuzhang Shang² Emilio Ferrara¹

¹University of Southern California ²University of Central Florida

ABSTRACT

Long-horizon robot manipulation chains many contact-rich skills into one multi-stage task. Vision-language-action (VLA) and world-action models (WAMs) increasingly master the individual skills. However, the chain itself still fails: errors compound beyond the policy’s ability to correct, and the execution of one subtask silently constrains the next. Coding agents offer a promising pathway: freeze the VLA and put an LLM agent in charge of it. The agent plans in language, moves in free space with analytic primitives, and invokes the VLA only for the contact-rich segments, writing all adaptation into language memory. Yet applied directly to long horizons, the agent-plus-VLA recipe breaks twice. ❶ Its competence is acquired through whole-task exploration at test time, whose cost is multiplicative in the number of stages: where a single stage requires T episodes on average, a K -stage task requires on the order of T^K , and a failed episode does not identify the stage responsible. ❷ It has no representation of the transitions: the VLA primitive carries an exit condition but no entry condition, and a subtask can succeed in a form its successor cannot use. We present **BATON**, which addresses the two failures in turn.¹ Against ❶, **BATON** makes the subtask the unit of exploration. Each subtask is explored in the inexpensive short-horizon regime, and its solution is stored in memory. A long-horizon trajectory is then composed from these solutions rather than discovered whole. Exploration cost thus becomes additive ($T \cdot K$), and every failure is attributed to a single stage. Against ❷, **BATON** equips this exploration with a transition-aware memory. Within a subtask, a verifier agent governs the *invocation transition*: the VLA is invoked only after the wrist view confirms that the scene is ready. Across subtasks, *handoff transition* restores an entry state disturbed by the predecessor’s residue, and *lookahead transition* determines the execution strategy whose outcome the successor can inherit. No parameters are updated anywhere. On the long-horizon manipulation benchmark RoboMemArena, **BATON** improves task success rate by 11.6% and cumulative success rate by 14.9% over the current SoTA.

1 INTRODUCTION

What separates current manipulation policies from generalist ones is less about any single skill than about composition: interpreting an instruction, then holding it together across many contact-rich steps whose errors compound. The skills themselves are increasingly within reach: vision-language-action (VLA) models and world-action models (WAMs) map observations and language directly to low-level actions, generalizing broadly through web-scale pretraining [3, 15, 5, 2, 12, 1, 30, 22, 36]. What none of them yet delivers is the chain. Set $\pi_{0.5}$ [12] on each subtask in isolation (open a drawer, pick up a can, place a can in the drawer) and it learns and executes cleanly; set it on the thousand-step task that chains those same subtasks and it completes half of the intermediate stages while finishing only 10% of the full tasks [18]. The stages succeed, and the task dies between them.

¹The recipe needs this interface, not a particular backend: a VLA, a WAM, or any contact-rich policy callable the same way, slots in unchanged. We write VLA throughout for simplicity.

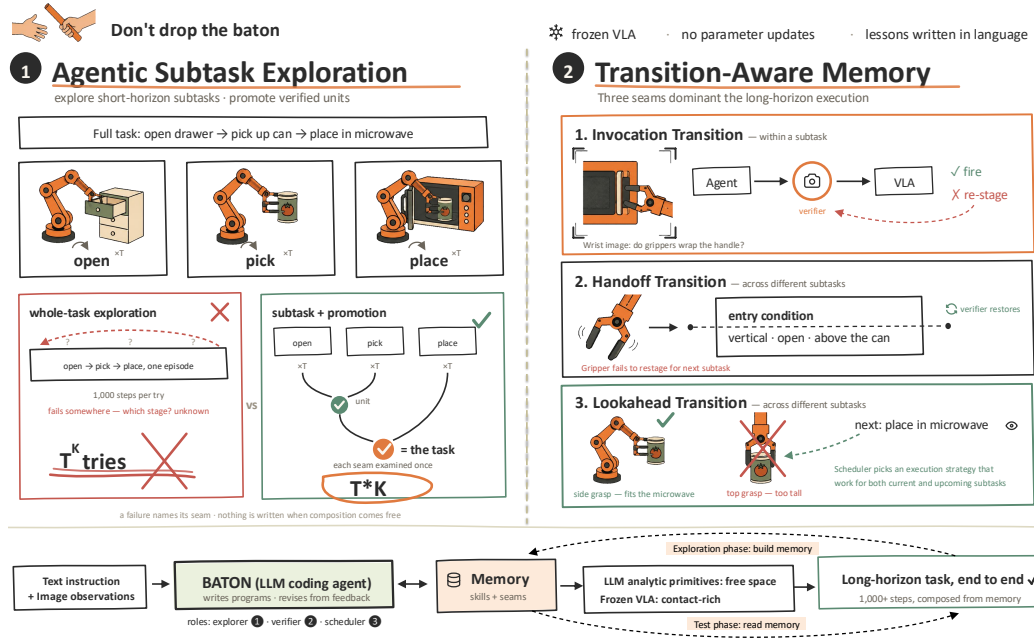


Figure 1: **BATON makes every transition of a long-horizon task a first-class object.** *Bottom:* an LLM coding agent writes programs over analytic primitives, calls the frozen VLA only for contact, and stores every lesson in language memory; no parameters are updated. **1 Agentic subtask exploration** (left): whole-task exploration pays T^K episodes and cannot say which stage broke; **BATON** explores each subtask short-horizon and promotes verified units round by round, so cost stays additive and a failure names its stage. **2 Transition-aware memory** (right): three transitions, one store. The *invocation transition* fires the VLA only after the wrist view confirms the scene is ready; the *handoff transition* repairs the entry state a predecessor’s residue disturbs; the *lookahead transition* lets the next subtask dictate how the current one is executed. Transitions live on the edges of the store.

Two things go wrong. First, errors compound and the policy cannot repair them. End-to-end VLAs overfit the demonstrated trajectories from their curated initial states [7, 8]. When each stage starts wherever the previous one ended, it can only replay memorized motions from states it has never seen, and it can neither recognize nor correct the accumulating drift [27, 41]. Second, neighboring subtasks are coupled through execution: how one subtask is carried out fixes the state the next begins from, and an execution that succeeds locally can leave the next subtask impossible [17, 39, 9].

Coding agents are built for exactly this kind of composition [38, 37, 31]. In this line of work [20, 10], an LLM agent controls the robot by writing programs over perception and control primitives. The programs run, and the agent revises them from execution feedback. Nothing is trained; the agent improves by trying things at test time. Harness VLA [40] brings this recipe to contact-rich manipulation by adding a frozen VLA to the agent’s toolbox. The agent plans in language and moves through free space with analytic, inverse-kinematics-solved primitives; where the task turns contact-rich, it hands over to the VLA, wrapped as a retryable primitive with a prompt and a stop predicate. For long-horizon control, this division of labor is attractive on three counts. First, each part does what it is best at: the VLA handles local contact, and the agent handles composition, semantic re-binding, and staging. Second, nothing is fine-tuned. One frozen policy serves every task. Third, everything the system learns is written in language, so it can be read, audited, and reused. On short-horizon tasks, this design already works: skills stay reliable under perturbations that break end-to-end VLAs [40]. Long-horizon manipulation is another matter: no coding agent yet earns the same competence over a thousand-step chain.

Two difficulties block the way. **1** is the large cost of full task exploration. The agent earns its competence at test time. During exploration, the agent freely trials staging orders, pre-contact poses, VLA invocation timings, and termination thresholds until the task first succeeds, then consolidates the working solution into memory [40]. For a short-horizon task of a hundred-odd steps, first success arrives after T exploration episodes on average, with a matching bill of planner tokens: a real price, but one paid once per task. Stretching the same procedure over a task of K such stages breaks it. An episode succeeds only if every stage does, so whole-task exploration needs on the order of T^K episodes, beyond any budget (see Fig. 1, left). To make matters worse, the failures are uninformative:

an episode reveals only that the task failed, not which stage broke. ❷ is the transitions the chain must cross. Beyond its cost, exploration is uninformed precisely where a chain is fragile: at the crossings. Within a subtask, no signal indicates when the scene is ready for the contact-rich policy, so the moment of handover must be found by repeated trial. Across subtasks, individually successful skills fail to compose, for two reasons. First, a completed subtask alters the state its successor starts from. A subsequent failure is therefore ambiguous: the successor's skill may be inadequate, or its entry state is merely wrong, however exploration cannot distinguish the two. Second, even a correct entry is not sufficient: subtasks are interdependent, and the requirements of the next constrain how the previous one should be executed. This dependency spans subtasks, so a learner that acquires them one at a time can neither observe it nor correct for it.

Targeting ❶, we propose **BATON**, an LLM agent orchestrating a frozen VLA for long-horizon robot manipulation tasks. **BATON** makes the subtask the unit of exploration: it decomposes a long-horizon task and explores each subtask in the cheap short-horizon regime where the recipe can work. Each solved subtask's successful execution is extracted into abstract memory. A long-horizon trajectory is then composed from these pieces by memory rather than discovered whole. Exploration cost now scales additively in the number of new subtasks and boundaries rather than multiplicatively (see Fig. 1, left). A failure names the stage that caused it instead of voiding a thousand-step episode. A relay team does not train by running ten thousand full races; it drills each exchange.

BATON meets ❷ with a *transition-aware* memory that consists of three parts. *Within* a subtask lies the *invocation transition*, where control passes from the agent to the contact-rich VLA after a *verifier agent* confirms. *Across* subtasks, two transitions govern each boundary. The *handoff transition* concerns the entry state of the successor: the boundary records an entry condition, and the verifier restores the scene to satisfy it before the next subtask begins. The *lookahead transition* operates in the reverse direction: the requirements of the subtask that follows determine how the current one is executed. A *scheduler agent* enforces this dependency, selecting from memory the strategy whose outcome the successor can inherit. Together these conditions turn each boundary into a *handoff contract* that is drafted in language, checked at execution, and stored with the coarse trajectories it connects. A good decomposition cuts a long task at interfaces that can be checked.

Our core contributions are:

- We present **BATON**, the first coding agent to carry long-horizon robot manipulation end to end over a frozen VLA, learning by test-time exploration with no parameter updates.
- **BATON** rests on two mechanisms: *hierarchical subtask exploration* for ❶, which keeps exploration cost additive; *transition-aware memory* for ❷, which attaches checkable conditions to the invocation, handoff, and lookahead transitions.
- On RoboMemArena's thousand-step household tasks, **BATON** improves task success by 11.6 points and cumulative success by 14.9 points over the current SoTA.

2 RELATED WORK

BATON builds on three lines of work: long-horizon robot manipulation, memory for manipulation agents, and agentic control of frozen VLAs. Its novelty does not lie in any one of these components in isolation. It lies in their combination through an object none of them represents: the transition between subtasks, which **BATON** explores one at a time, checks at run time, and stores as the edges of its memory.

Long-horizon robot manipulation. The dominant response to long horizons is to train for them. Web-scale VLA pretraining keeps improving the individual skills [3, 15, 2, 12], but the resulting policies are reactive: they predict the next action from the current observation, overfit the curated initial states of their demonstrations [7, 8], and cannot correct their own compounding drift [27, 41]. Memory-augmented VLAs train history into the policy [29, 32], and memory-oriented benchmarks measure what remains: on RoboMemArena's thousand-step household tasks, every reported policy completes far more stages than full tasks [18, 6]. That skills fail when chained where each succeeds alone is documented across settings. Sequenced policies need terminal states aligned with what the next policy expects [17]; BOSS attributes 34–67% of chained-task failure to observation-space shift at subtask boundaries, which more demonstrations do not remove [39]; VLA skills with isolated

success rates of 77–100% still collapse under composition [28]; and nine grasping policies with near-identical first-subtask success (about 95%) spread from 0% to 83% on the subtask that follows [9]. The fixes, again, train for the boundary: **HANDBFUL** selects among finger configurations by curriculum-based successive elimination [9], and **Foresight Residual RL** shapes terminal states with a residual policy rewarded by a learned downstream-success predictor [21]. Both bind the lesson to a fixed successor policy in a known sequence and pay a per-boundary training cost, so every new pairing of skills pays again. **BATON** encodes the same insight symbolically, in the manner of a coding agent. The handoff contract is drafted in language at plan time, checked at the boundary, and corrected when it fails. Because it attaches to the transition rather than to the task, it is paid for once and reused by every task that crosses the same boundary.

Memory, skill libraries, and task-aware grasping. Skill libraries since *Voyager* converge to one program per skill name [37]. Embodied-memory systems for manipulation key retrieval on the current instruction, recent observations, or execution history [35, 13]. Memory-augmented VLAs build episodic context into the policy itself [29] or retrieve past keyframes from an experience buffer [32]; in either case the memory records what was seen, not which strategy was chosen. *HiMe* is the sharpest foil. Its hierarchy indexes control frequency, it deletes conflicting entries to keep memory consistent, and its subtask monitor records *that* a grasp succeeded rather than *which* grasp [13]. The rival strategies we preserve are, in that design, a defect to remove; yet which grasp was taken is exactly what the next subtask depends on. Task-oriented grasping selects a grasp for a named use, classically from a fixed semantic taxonomy [24] and more recently with open-vocabulary use descriptions grounded by foundation models [34]; either way, the condition is a use named for one object, not the plan that remains. Task-and-motion planning derives grasp constraints by reasoning backward over symbolic operators [14], which requires engineered predicates and models [11]. To our knowledge, **BATON**'s memory is the first to keep rival strategies per subtask and to key retrieval on the remaining plan, through the scheduler that decides *how* the current subtask should be executed. The handoff contract, drafted and refined in language without training, plays the role that taxonomies, predicates, and value functions play in these lines.

Agentic control of frozen VLAs. An LLM agent can control a robot by writing programs over perception and control primitives, revising them from execution feedback with nothing trained [20, 10]; *Harness VLA* extends this recipe to contact-rich manipulation by wrapping a frozen VLA as a retryable primitive [40] (Sec. 3.1). *ASPIRE* runs agentic skill discovery in code, with no learned policy in the loop [23]. Adjacent lines place a reasoning agent above the motor level without writing programs. VLM planners orchestrate VLAs as callable tools [19, 4]; dual-system models couple a slow reasoner to a fast policy [30, 12, 25]; world-action models extend the motor level with predictive world modeling [22, 36]. All of these systems make individual skill invocations reliable, or discover new skills. None constrains what one invocation must leave behind for the next, and none reports what its exploration costs, a cost that multiplies once a task spans many stages. **BATON** keeps the substrate of *Harness VLA* and treats the transition itself as the unit of both exploration and learning.

3 METHOD

We first lay out the background our design rests on: what end-to-end VLAs do well and where they fail, and the coding-agent recipe that harnesses a frozen VLA (Sec. 3.1). We then state why this recipe does not stretch over a long horizon (Sec. 3.2). Our method follows in two steps: Sec. 3.3 decomposes the task and explores it hierarchically, subtasks first, then verified units chained outward to the full chain; Sec. 3.4 develops the transition-aware memory, within subtasks and across them, that this exploration writes and execution reads.

3.1 PRELIMINARIES

End-to-end VLA models on robot manipulation. A VLA policy maps raw observations and a language instruction directly to low-level actions. Trained by imitation on large corpora of contact-rich demonstrations, it excels precisely where analytic control is hardest: securing a grasp on irregular geometry, seating an object under tight clearance, or operating articulated fixtures such as drawers and faucets. This competence, however, is tied to the trajectory distribution of the training data, and it degrades sharply once deployment leaves that distribution. Robustness studies report success rates

falling from above 90% to below 30% under modest perturbations of viewpoint, layout, initial pose, or instruction phrasing [8, 42], and trace the fragility to a visual shortcut: when the instruction is predictable from the scene, the policy learns to replay memorized trajectory patterns from visual context and largely ignores the language input [7, 8]. A longer horizon exposes this limitation twice over. First, errors compound and the policy cannot repair them: each skill is cloned from curated initial states, yet over a chain every stage starts from whatever its predecessor leaves, so covariate shift accumulates [26, 27] while the policy replays memorized trajectories from states it never trained on, and success-only imitation gives it no signal to recognize the drift, let alone correct it [41]. Second, the subtasks are coupled through execution: how one stage is carried out fixes the state the next begins from and can make it outright infeasible [17, 39, 9], a dependency a policy trained stage by stage can neither see nor plan for. Dual-system designs [12, 1, 30] repair the *planning* half by construction: a high-capacity VLM decomposes the task, tracks progress, and issues one subtask at a time to the low-level policy. But the *execution* half persists: even given the correct subtask instruction, the policy must act from whatever state the preceding stages left behind, not from the curated initial states of its demonstrations, and no improvement at the planning level can correct a failure that lies below the planner's interface.

Coding agents for robot control. Coding agents attack the execution failure from the opposite direction: they restrict what the learned policy is asked to do. In this line of work, an LLM agent controls the robot by writing programs over perception and control primitives [20, 10]. The programs do not run bare: they run inside a harness that gives the agent access to its tools, checks on intermediate results, an execute-and-observe loop, and a memory that survives across attempts, so that a failed step can be retried, diagnosed in language, and its eventual fix written back for reuse [38, 37, 31]. For manipulation, the reach of such an agent is limited by its primitives. Motions that admit closed-form treatment, such as transporting the end-effector along an inverse-kinematics-solved path, reorienting the wrist, or actuating the gripper, are exactly the easy part of a task; the contact-rich part is what no scripted primitive covers. Harness VLA [40] resolves this by giving the agent a single learned primitive alongside the analytic ones: `VLA_ACT`, a frozen VLA wrapped with a task-conditioned prompt and a stop predicate, so that it can be invoked, monitored, and retried like any other tool. The agent composes the analytic primitives for everything that does not involve contact, from locating the target and staging the approach to transport and release, and hands control to `VLA_ACT` only when contact begins. A skill under this division factors into two segments,

$$u = \underbrace{a}_{\text{non-contact: LLM agent}} \triangleright \underbrace{v}_{\text{contact-rich: VLA_ACT}}, \quad (1)$$

every step that requires following an instruction belonging to the LLM, which follows instructions reliably, while the VLA acts only in a short contact window that opens from a pre-contact pose the agent has already staged, that is, from a state near its training distribution. Each component is consulted only where it is strong. Note that Eq. (1) carries no symbol for the crossing itself: where a ends and v begins is represented nowhere. The agent discovers it by test-time exploration on a reference instance of the task, trialing invocation timings until one happens to work, and consolidates the finding into language memory together with the other orchestration choices, the staging order, the pre-contact pose, the stop threshold [40]. On short-horizon tasks the resulting skills stay reliable under the perturbations that break end-to-end VLAs; we therefore adopt the harnessed frozen VLA, rather than the bare policy, as our substrate.

Long-horizon manipulation. A long-horizon task $\mathcal{T}$ is an instruction (set the table, clean the kitchen) whose execution decomposes into an ordered chain $\langle u_1, \dots, u_K \rangle$ of subtasks, each a short-horizon, contact-rich skill of a hundred-odd control steps (open the drawer, pick up the can, place the can in the drawer), with the full chain running to thousands of steps. Two properties distinguish the chain from its parts. First, the subtasks are coupled through state: the initial state of u_{k+1} is not sampled from a curated distribution but is whatever terminal state u_k produces, carrying the residue of every preceding stage. Second, success is conjunctive: $\mathcal{T}$ succeeds only if every u_k does, so per-stage reliability compounds multiplicatively across the chain. It is exactly this regime in which competent skills fail to add up: a state-of-the-art VLA on thousand-step household tasks completes roughly half of the intermediate stages it encounters yet only $\sim 10\%$ of full tasks [18].

3.2 PROBLEM STATEMENT

The competence described in Sec. 3.1 is bought by search. On a reference instance of a task, the agent trials staging orders, pre-contact poses, invocation timings, and termination thresholds until the task first succeeds, and only then writes the working solution to memory [40]. For a short-horizon task this search is affordable, and its cost is paid once. The question is what happens when the same procedure is applied to a chain of K subtasks. Two obstacles stand in the way.

The ❶ obstacle is the cost of exploration. Success over a chain is conjunctive, so whole-task exploration must find a working realization of every stage within a single episode. If one stage reaches its first success after T exploration episodes on average, then K stages explored jointly require on the order of T^K episodes, because each episode gambles on all stages at once. The structure of the search makes matters worse. Before an episode can work on stage k , it must first traverse stages 1 through $k - 1$. Most of its steps are therefore spent re-crossing ground that earlier episodes already covered, and one upstream slip ends the attempt before the stage under study is even reached. Failure also teaches almost nothing. An unsuccessful episode reports little beyond the fact that the chain broke somewhere, and the memory has no unit in which to bank a partial result: what the recipe consolidates is a whole-task trace, so three solved stages followed by one failed stage leave nothing behind. Exploration that is cheap and self-correcting on one subtask becomes, over a chain, expensive, blind, and unable to retain its own progress.

The ❷ obstacle is the transitions. The recipe specifies each subtask internally but leaves its crossings unspecified: neither the point at which control passes to the frozen VLA, nor the state in which one subtask should deliver the scene to the next. The first crossing is the invocation of the frozen VLA. `VLA_ACT` carries a stop predicate, an exit condition, but no counterpart on entry. In single-task use this absence is inconsequential: every episode begins from the same reset state, so a workable invocation timing can be found by trial and then reused. Over a chain, the state at each boundary is produced by the preceding stage and varies across runs, so no fixed timing remains valid. If the VLA is invoked before the scene is adequately staged, it begins from a state outside its training distribution, and the transition readmits precisely the out-of-distribution failure the harness is designed to prevent. The second crossing is the boundary between subtasks, where the recipe is deficient in two respects. A subtask begins from the terminal state of its predecessor, which may carry residue the stored skill was not trained to accommodate: a gripper still rotated from opening a door. The resulting failure is ambiguous—the successor’s skill may be inadequate, or its entry state may be incorrect—and exploration cannot distinguish the two cases, so the budget may be spent re-exploring a skill that is not responsible for the failure. Moreover, a correct entry state is not sufficient, because a local success predicate admits many realizations: a drawer opened just widely enough to satisfy the predicate may remain too narrow for the hand that must subsequently reach into it. The recipe consolidates whichever realization succeeds first, records nothing about which realization it was, and never consults the subtask that follows; whether the stored strategy is compatible with its successor is left to chance.

Together the two obstacles fix the shape of our method. Exploration must be priced per subtask, so that cost adds across the chain rather than multiplying and a failure names the stage that caused it (Sec. 3.3). And the transitions must stop being by-products of whichever rollout happened to succeed first: both the crossing inside a subtask, where control passes to the frozen VLA, and the boundary between subtasks, where one stage hands the scene to the next, must become conditions the system states, checks, and remembers (Sec. 3.4).

3.3 HIERARCHICAL SUBTASK EXPLORATION

BATON is a coding agent over the harnessed frozen VLA of Sec. 3.1: analytic staging primitives, plus `VLA_ACT` as the single learned primitive; any contact-rich policy exposing the same call could serve as the backend. All roles described below are played by one LLM agent, and no parameters are updated anywhere in the system. Two designs follow. This section scopes exploration to one subtask at a time and composes verified units outward, boundary by boundary, so that cost stays additive and failures stay attributable; Sec. 3.4 then equips this exploration with a transition-aware memory, one entry for each of the three transitions the chain must cross, written during exploration and read at execution.

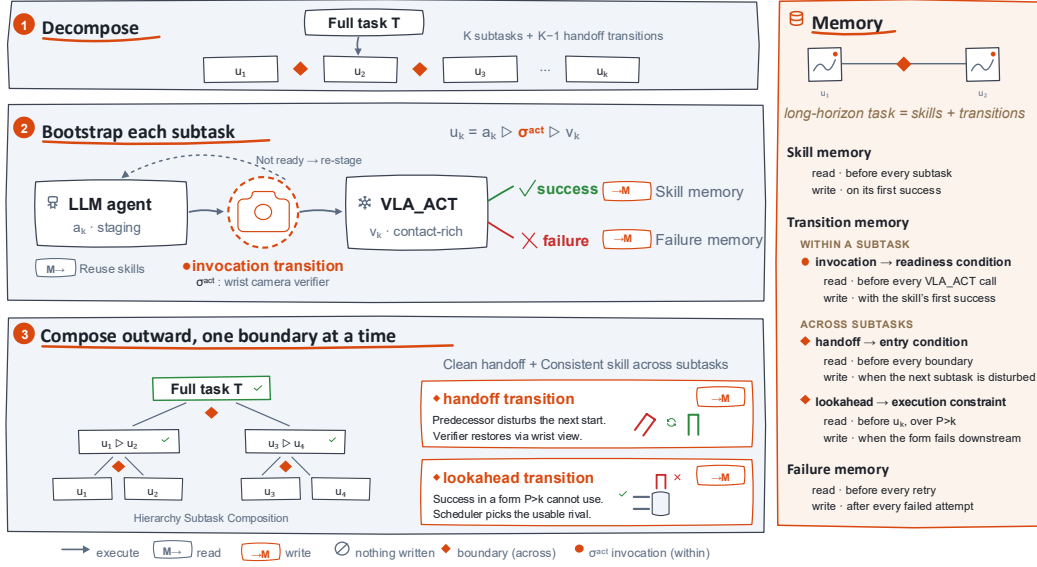


Figure 2: **BATON overview.** *Left:* The agent drafts a long-horizon instruction into K subtasks joined by $K-1$ handoff boundaries (Eq. (2)). Each subtask is explored through bootstrapping and then composed hierarchically into the full task. Within a subtask, the agent explores candidate skills and triggers an *invocation transition*, in which a wrist-camera verifier confirms that the scene is ready before VLA_ACT is called to perform contact-rich behavior. Across subtasks, a *handoff transition* ensures that the terminal state of one subtask does not disturb the initial state of the next, while a *lookahead transition* propagates the execution strategy of the current subtask to correlated successors. *Right:* Memory comprises three components. All three are involved during exploration, whereas only the skill and transition memories are read at test time.

Decompose first. Given a long-horizon instruction, **BATON** never explores the task whole. The agent first drafts a decomposition into subtasks and the boundaries that join them,

$$\mathcal{T} = u_1 \triangleright \sigma_{1,2} \triangleright u_2 \triangleright \sigma_{2,3} \triangleright \dots \triangleright \sigma_{K-1,K} \triangleright u_K, \quad (2)$$

where each subtask u_k is a short-horizon skill and each $\sigma_{k,k+1}$ is the *handoff boundary* through which u_k passes the scene to u_{k+1} . Drafting consults the memory of Sec. 3.4, so subtasks and boundaries already crossed in earlier tasks are inherited rather than rediscovered. The decomposition answers the first obstacle of Sec. 3.2 outright and gives the second one its objects. Exploration is priced per piece: *if a single subtask reaches its first success after T episodes on average, the chain costs on the order of KT short episodes, plus the composite runs of the rollout below, rather than T^K long ones.* And blame is localized: a failure now happens inside a named subtask or at a named boundary, where it can be diagnosed and repaired, instead of voiding a thousand-step episode.

For each subtask without a usable memory entry, the Explorer runs the bootstrapping loop of Zhang et al. [40] scoped to that subtask, trialing staging orders, pre-contact poses, and VLA_ACT invocations, the latter under the wrist-camera gate of Sec. 3.4. Each failed attempt is written to *failure memory*: what was tried and how it failed, so later attempts steer away from repeated mistakes. The first success is consolidated into *memory* as a coarse trajectory: the staging waypoints, the readiness condition of the invocation transition (Sec. 3.4), the VLA_ACT call, and a termination check, all written as symbolic perception queries rather than coordinates. Exploration of a subtask therefore never leaves the short-horizon regime, and the multiplicative cost of Sec. 3.2 never forms.

Hierarchical composition. Solved subtasks do not yet make a chain, and **BATON** does not jump from one to the other: composition is verified outward, level by level. Related neighbors are chained first: open the top drawer, then close it. A pair that succeeds is trusted as a unit from then on and chained in its turn, with the middle-drawer unit, larger compositions with each other, until what runs end to end is the task itself. The rollout adds no skills of its own: a unit is not a new entry but its parts executed in order, and when two skills compose cleanly, nothing is written. Memory grows only where composition does not come free: a boundary whose handoff disturbs the next skill's start,

or whose later subtasks constrain how an earlier one must be executed, earns an entry of its own (Sec. 3.4). Every round chains compositions that are already verified, so its search falls only on the boundary where the two sides meet; each of the $K - 1$ handoff boundaries is examined once, in the round where its sides first meet, and this stage stays additive as well. A failure likewise surfaces at the boundary just created, never inside a verified composition; a boundary counts as crossed when the successor succeeds from the state its predecessor leaves. What the rollout finds at those boundaries, and what must be remembered about them, is the subject of Sec. 3.4.

3.4 TRANSITION-AWARE MEMORY

What makes the memory long-horizon is not what it stores about subtasks but what it stores about transitions, within each subtask and across them, and these are of three kinds. *Within* each subtask lies the *invocation transition*, where control crosses from the agent to the frozen VLA. *Across* subtasks, the rollout of Sec. 3.3 exposes two distinct failures at each handoff boundary, one in each direction: in the *handoff transition* the predecessor disturbs the entry state the next skill starts from, and in the *lookahead transition* the successor constrains how the current skill must be executed. The memory keeps one kind of entry for each of the three; we take them in turn, then the store that unifies them.

Within a subtask: the invocation transition. Where Eq. (1) carries no symbol for the crossing between agent and policy, **BATON** writes it into the factorization as an explicit *invocation transition*:

$$u_k = \underbrace{a_k}_{\text{non-contact: LLM agent}} \triangleright \sigma_k^{\text{act}} \triangleright \underbrace{v_k}_{\text{contact-rich: VLA_ACT}}. \quad (3)$$

The transition σ_k^{act} is made conditional rather than searched: VLA.ACT may fire only when a stated readiness condition holds on the wrist camera, judged from the live wrist image by an agent role we call the *verifier*. For “open the microwave door”, the two gripper fingers must show on either side of the handle before pulling; for placing into a basket or a drawer top, both sides of the gripper must show inside the container’s four edges before release. On failure the agent re-stages with analytic primitives and checks again; an attempt that cannot be made ready is recorded in failure memory, never forced through the gate. The condition is stored with the skill’s coarse trajectory, and it pays twice. During exploration, no episodes are wasted discovering by failure that the VLA fired too early: a discovered timing is a fact about one successful rollout, while a stated condition steers the search and then, unchanged, guards every later execution. During execution, it catches drift: the fine operations that are reliable in isolation (opening the microwave, releasing into the basket) are exactly the ones we observe failing late in long runs, and the gate stops the wandered pre-contact state at the transition instead of passing it into contact.

Across subtasks (i): the handoff transition. The handoff transition answers the forward failure: the entry state a skill starts from has been moved by its predecessor. After opening the microwave door, the gripper is left rotated toward the horizontal; carried unchanged into “pick up the can”, the tilted hand closes beside the can it grasps cleanly from its own reset. Neither skill is at fault; what moved is the entry state. Re-exploring either would find nothing to fix, and the successor’s own staging cannot help either: its coarse trajectory was consolidated where no re-orientation was ever needed. What exploration never met, the skill cannot see. The boundary therefore records an *entry condition*: what the wrist view must show for u_{k+1} to start cleanly, here the gripper restored to vertical, open, and above the can. It lives on the edge rather than on either node because it is a fact about the pair: which residue arrives depends on the predecessor, and which of it matters depends on the successor. The same verifier enforces it at the boundary, restoring the state with analytic primitives (re-orient, re-open, re-stage) before the next skill starts. Classical skill formalisms attach an initiation set to each skill [33], learned offline from experience [16]; here the condition is written in language on the boundary and checked zero-shot on live observations.

Across subtasks (ii): the lookahead transition. The lookahead transition answers the backward failure: a skill succeeds, but in a form its successor cannot use. “Pick up the tomato can” admits rival strategies: grasp from the side; pinch the rim from the top; or push the fingers from the top all the way down, so the whole can sits in the palm. Which rival is right is not a property of the subtask. Bound next for the microwave, only the side grasp works, since gripper plus can under a top grasp exceed the interior height; bound for a long transport with repeated pouring, the top

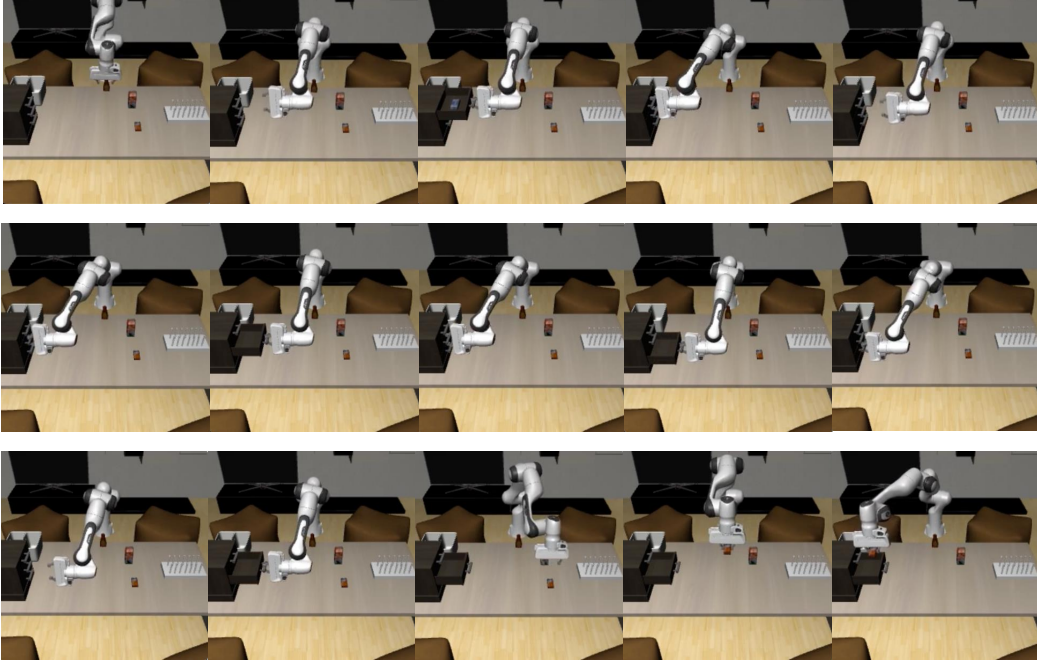


Figure 3: **Demo of a successful long-horizon RoboMemArena task execution during testing.** This task runs over **1800 steps**, roughly 13 times the length of a typical LIBERO task. It requires opening and closing three drawers in order, then reopening the empty drawer and placing the butter into it. During the exploration phase, only subtasks and their transitions are explored; the full task is composed from memory and succeeds on the held-out seed.

grasp must go all the way down, since a shallow rim pinch drops the can mid-pour. The boundary therefore records an *execution constraint*: under this successor, the current subtask must end in a stated form. A second agent role, the *scheduler*, enforces these. Before u_k executes, it reads the remaining plan $\mathcal{P}_{>k} = \langle u_{k+1}, \dots, u_K \rangle$, collects the constraints its boundaries induce, and retrieves the rival that satisfies them, so the same query “pick up the can” returns the side grasp when a microwave placement follows and the deep top grasp when transport and pouring do. When no stored rival fits, the subtask returns to the Explorer with the constraint added to its goal.

One store for the three transitions. The memory $\mathcal{M}$ integrates the three kinds of entry by mirroring Eq. (2): nodes and edges. A node holds a subtask’s failure entries and its rival coarse trajectories, each embedding its own readiness condition at the invocation transition. An edge holds, for the boundaries that earned one, the entry condition of the handoff transition and the execution constraints of the lookahead transition; together they form the boundary’s *handoff contract* $\mathcal{C}_{k,k+1}$, drafted in language with the chain, checked at execution, and revised when it fails. Rivals are not enumerated in advance: a first success enters alone, and a new rival is added exactly when a constraint rules the incumbent out and re-exploration under it succeeds; the incumbent is kept and annotated, so the set grows with the boundaries the subtask has served. Edges are keyed by transition type rather than by task, so “release into a container with a narrow lip” transfers from baskets to bins; and when execution violates a contract, the violation is written back and the offending strategy ruled out under it, so the lesson attaches to the transition, is paid for once, and is inherited by every later task that crosses the same transition. This write-back is what keeps exploration additive as the task set grows. Prior skill libraries keep one canonical strategy per skill and key retrieval on the current command or context [37, 35, 13, 40]; collapsing rivals is precisely what blinds an executor at the boundary, because which rival is best is written on the edge, not the node. And relative to task-oriented grasping [24] and foresight values trained by reinforcement learning [21, 9], this memory needs no taxonomy and no gradients: its conditions and constraints are corrected when they fail and transfer to task sequences never seen together, because they attach to transitions, not tasks.

Table 1: RoboMemArena [18]: per-category and average TSR/CSR (%). Rows in the top group are reported by the benchmark; rows in the middle group we run ourselves. The ground-truth oracle executes with perfect memory of the past and bounds what retrospection alone can achieve; it is excluded from bolding.

Method	Transferring		Occlusion		Counting		Sequence		Average	
	TSR	CSR	TSR	CSR	TSR	CSR	TSR	CSR	TSR	CSR
GPT-5.4 (VLM only)	13.8	32.9	1.8	9.2	12.9	50.7	15.0	47.3	8.7	30.5
$\pi_{0.5}$ [12]	20.0	42.8	12.7	17.2	14.3	50.9	60.0	71.6	21.5	38.7
HiF-VLA	17.5	38.9	12.7	27.1	8.6	45.9	42.5	70.2	16.9	39.8
MemoryVLA [29]	15.0	37.2	7.3	13.1	14.3	55.1	37.5	65.2	15.0	35.3
MemER [32]	20.0	36.1	16.4	33.2	27.1	65.1	65.0	79.1	27.3	49.1
PrediMem [18]	22.5	45.2	27.3	38.4	45.7	69.3	72.5	89.5	38.5	55.2
<i>PrediMem + GT memory (oracle)</i>	32.5	54.8	33.6	49.8	51.4	75.6	85.0	92.3	46.1	64.8
FrameSamp+Modul [6]	63.8	72.1	39.1	56.5	31.4	57.8	73.8	86.9	46.1	63.9
Harness VLA [40]	50.0	58.3	0.0	0.0	12.5	50.0	82.0	84.5	26.9	38.5
BATON (ours)	39.4	65.0	50.0	78.7	68.8	85.4	83.4	91.7	57.7	78.8

4 EXPERIMENTS

Our evaluation asks one question: does making the subtask the unit of exploration, and the transition the unit of memory, recover the composite-task performance that stage-level competence fails to predict (Tab. 1)?

Setup. We follow the Harness VLA configuration [40]: a frozen π_0 -family VLA orchestrated by an LLM agent. Our suite is *RoboMemArena* [18], described below. The protocol separates exploration from evaluation by scene configuration. For every task, all exploration runs on a single reference configuration, seed 50; evaluation runs on two held-out configurations, seeds 51 and 52, and every number we report averages the two. Baselines are of two kinds. *Reported by the benchmark* [18]: reactive $\pi_{0.5}$ [12]; the memory-augmented VLAs HiF-VLA, MemoryVLA [29], and MemER [32]; PrediMem, the benchmark’s own dual-system memory VLA, with and without ground-truth memory; FrameSamp+Modul [6]; and GPT-5.4 acting as a frozen VLM agent without a manipulation harness. *Run by us*: Harness VLA unmodified [40], exploring each task *whole* on the same seed 50, equipped with all of its own task-specific and global memory and given an exploration budget of 20 episodes per full task, then evaluated on the same held-out seeds. Metrics are the benchmark’s task success rate (TSR, all verification stages satisfied) and cumulative success rate (CSR, fraction of stages completed).

RoboMemArena. RoboMemArena evaluates long-horizon tasks whose completion depends on state carried *across* stages. Its 26 household tasks average over 1,000 steps and 3–9 verification stages in partially observable kitchen scenes, with 68.9% of subtasks memory-dependent; stage-level predicates make TSR and CSR measurable. Tasks fall into four categories: *transferring* (4 tasks), a source-to-target mapping among visually similar containers; *occlusion* (11), the locations and states of objects out of view; *counting* (7), how many times an action has already been performed; and *sequence* (4), a prerequisite subtask’s outcome that decides the next action. The benchmark frames these tasks as tests of remembering the past; the same chained, partially observable structure also concentrates transitions, and the *sequence* category is the coupling problem in benchmark form.

Results. **BATON** attains the best average performance in Tab. 1: 57.7% TSR and 78.8% CSR, exceeding the strongest reported system, FrameSamp+Modul [6], by 11.6 and 14.9 points respectively, and surpassing the ground-truth oracle (46.1 and 64.8) on both averages.

The reported rows locate the difficulty between the stages rather than within them. Every method completes a far larger fraction of stages than of tasks: $\pi_{0.5}$ converts a 38.7% CSR into a 21.5% TSR, and the oracle, which executes with a perfect record of the past, still converts 64.8 into 46.1. A retrospective memory is therefore not sufficient. Consistent with this reading, the memory-augmented executors gain chiefly on counting and sequence (MemER reaches 27.1 and 65.0 TSR against $\pi_{0.5}$ ’s

14.3 and 60.0), the categories in which recalling the past most directly determines the next action, while occlusion remains the hardest category for every reported method (the oracle itself reaches only 33.6 TSR). The frozen VLM agent without a manipulation harness fails almost entirely (8.7 TSR), which indicates that a reasoning layer alone does not close the gap either.

Our two rows compare the two ways of spending the same exploration budget. Whole-task Harness VLA reaches a first success only where the chain is short enough to bootstrap end to end: it performs strongly on sequence (82.0 TSR) but never solves an occlusion task during exploration, and its average (26.9 TSR) barely exceeds reactive $\pi_{0.5}$. **BATON** explores the same seed subtask by subtask and reaches 57.7% TSR, with the largest margins over the strongest reported system on occlusion (50.0 vs. 39.1) and counting (68.8 vs. 31.4). It also converts 73% of its completed stages into completed tasks, the highest ratio in the table, indicating that the remaining failures concentrate less at the boundaries. Transferring is the single category in which a reported system remains ahead; the deficit is largely an evaluation artifact, as the BDDL scene definition of one of the four transferring tasks (task 19) is faulty on the held-out seeds, so every rollout of that task scores zero regardless of execution. Fig. 3 shows one solved evaluation rollout.

5 CONCLUSION

Long-horizon manipulation has been treated as a problem of better skills, better planners, or better policies; we have argued it is, to a first approximation, a problem of *transitions*. Whole-task exploration that cannot say which stage broke, a contact-rich VLA invoked with no entry condition, and strategies kept for completing their subtask even when they block the next are all failures of an object no prior system represents: the transition. **BATON** makes that object explicit as a language-level contract, explores subtask by subtask so the multiplicative bill never comes due, clears each entry into the VLA from wrist-camera evidence, and schedules the present subtask by the ones still to come, on a frozen VLA, with no gradient anywhere and every lesson stored as a sentence someone can read.

REFERENCES

- [1] Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, et al. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.
- [2] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [3] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.
- [4] Siyi Chen, Hugo Hadfield, Alex Zook, Mikaela Angelina Uy, Chan Hee Song, Erwin Coumans, Xuning Yang, Faisal Ladhak, Qing Qu, Stan Birchfield, et al. Volo: A physical orchestrator for open-vocabulary long-horizon manipulation. *arXiv preprint arXiv:2606.07723*, 2026.
- [5] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems (RSS)*, 2023. *arXiv:2303.04137*.
- [6] Yinpei Dai, Hongze Fu, Jayjun Lee, Yuejiang Liu, Haoran Zhang, Jianing Yang, Chelsea Finn, Nima Fazeli, and Joyce Chai. RoboMME: Benchmarking and understanding memory for robotic generalist policies. *arXiv preprint arXiv:2603.04639*, 2026.
- [7] Yu Fang, Yuchun Feng, Dong Jing, Jiaqi Liu, Yue Yang, Zhenyu Wei, Daniel Szafrir, and Mingyu Ding. When vision overrides language: Evaluating and mitigating counterfactual failures in VLAs. *arXiv preprint arXiv:2602.17659*, 2026.

- [8] Senyu Fei, Siyin Wang, Junhao Shi, Zihao Dai, Jikun Cai, Pengfang Qian, Li Ji, Xinzhe He, Shiduo Zhang, Zhaoye Fei, et al. LIBERO-plus: In-depth robustness analysis of vision-language-action models. *arXiv preprint arXiv:2510.13626*, 2025.
- [9] Ethan Foong, Yunshuang Li, Hao Jiang, Gaurav S. Sukhatme, and Daniel Seita. Handful: Sequential grasp-conditioned dexterous manipulation with resource awareness. *arXiv preprint arXiv:2604.25126*, 2026.
- [10] Letian Fu, Justin Yu, Karim El-Refai, Ethan Kou, Haoru Xue, Huang Huang, Wenli Xiao, Guanzhi Wang, Dantong Niu, Fei-Fei Li, Guanya Shi, Jiajun Wu, Shankar Sastry, Yuke Zhu, Ken Goldberg, and Linxi Fan. CaP-X: A framework for benchmarking and improving coding agents for robot manipulation. *arXiv preprint arXiv:2603.22435*, 2026.
- [11] Caelan Reed Garrett, Rohan Chitnis, Rachel Holladay, Beomjoon Kim, Tom Silver, Leslie Pack Kaelbling, and Tomás Lozano-Pérez. Integrated task and motion planning. *Annual Review of Control, Robotics, and Autonomous Systems*, 2021.
- [12] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- [13] Li Ji, Siyin Wang, Pengfang Qian, Xiaopeng Yu, Yihai Tian, Zhaoye Fei, Jingjing Gong, and Xipeng Qiu. Hime: Hierarchical embodied memory for long-horizon vision-language-action control. *arXiv preprint arXiv:2607.03449*, 2026.
- [14] Leslie Pack Kaelbling and Tomás Lozano-Pérez. Hierarchical task and motion planning in the now. In *2011 IEEE International Conference on Robotics and Automation (ICRA)*, 2011.
- [15] Moo Jin KKim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. OpenVLA: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- [16] George Konidaris and Andrew Barto. Skill discovery in continuous reinforcement learning domains using skill chaining. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2009.
- [17] Youngwoon Lee, Joseph J. Lim, Anima Anandkumar, and Yuke Zhu. Adversarial skill chaining for long-horizon robot manipulation via terminal state regularization. *arXiv preprint arXiv:2111.07999*, 2021.
- [18] Huashuo Lei, Wenxuan Song, Huarui Zhang, Jieyuan Pei, Jiayi Chen, Haodong Yan, Han Zhao, Pengxiang Ding, Zhipeng Zhang, Lida Huang, Donglin Wang, Yan Wang, and Haoang Li. Robomemarena: A comprehensive and challenging robotic memory benchmark. *arXiv preprint arXiv:2605.10921*, 2026.
- [19] Zixing Lei, Changxing Liu, Yichen Xiong, Minhao Xiong, Yuanzhuo Ding, Zhipeng Zhang, Weixin Li, and Siheng Chen. Towards long-horizon embodied agents with tool-aligned vision-language-action models. *arXiv preprint arXiv:2605.13119*, 2026.
- [20] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 2023.
- [21] Yuhan Liu, Xinyu Zhang, Litao Liu, and Abdeslam Boularias. Foresight residual rl for long-horizon robot manipulation with vision-language-action models. *arXiv preprint arXiv:2607.16506*, 2026.
- [22] Yushan Liu, Peibo Sun, Shoujie Li, Yifan Xie, Lingfeng Zhang, Xintao Chao, Shiyuan Dong, Fang Chen, Xiao-Ping Zhang, and Wenbo Ding. OA-WAM: Object-addressable world action model for robust robot manipulation. *arXiv preprint arXiv:2605.06481*, 2026.

- [23] Runyu Lu, Yubo Wu, Ethan Kou, Letian Fu, Wenli Xiao, Ajay Mandlekar, Yinzen Xu, Guanya Shi, Ken Goldberg, Ang Chen, et al. Aspire: Agentic skills discovery for robotics. *arXiv preprint arXiv:2607.00272*, 2026.
- [24] Adithyavairavan Murali, Weiyu Liu, Kenneth Marino, Sonia Chernova, and Abhinav Gupta. Same object, different grasps: Data and semantic knowledge for task-oriented grasping. In *Proceedings of the 4th Conference on Robot Learning (CoRL)*, 2021.
- [25] Jiaqi Peng, Xiqian Yu, Delin Feng, Yuqiang Yang, Wenzhe Cai, Jing Xiong, Ganlin Yang, Jinliang Zheng, Jiafei Cao, Xueyuan Wei, et al. Cortex: A bidirectionally aligned embodied agent framework for long-horizon manipulation. *arXiv preprint arXiv:2607.05377*, 2026.
- [26] Stéphane Ross and Drew Bagnell. Efficient reductions for imitation learning. In *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics. JMLR Workshop and Conference Proceedings*, 2010.
- [27] Stéphane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *International Conference on Artificial Intelligence and Statistics*, 2011.
- [28] Ke Rui, Yushen Zuo, Jiawei Wang, Haoran Jia, Jinming Ma, Weitao Zhou, and Minglei Li. Diagnosing semantic handoff failures in agent-orchestrated vision-language-action skill composition. *arXiv preprint arXiv:2607.06256*, 2026.
- [29] Hao Shi, Bin Xie, Yingfei Liu, Lin Sun, Fengrong Liu, Tiancai Wang, Erjin Zhou, Haoqiang Fan, Xiangyu Zhang, and Gao Huang. MemoryVLA: Perceptual-cognitive memory in vision-language-action models for robotic manipulation. In *International Conference on Learning Representations (ICLR)*, 2026.
- [30] Lucy Xiaoyang Shi, Brian Ichter, Michael Equi, Liyiming Ke, Karl Pertsch, Quan Vuong, James Tanner, Anna Walling, Haohuan Wang, Niccolo Fusai, et al. Hi robot: Open-ended instruction following with hierarchical vision-language-action models. *arXiv preprint arXiv:2502.19417*, 2025.
- [31] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023.
- [32] Ajay Sridhar, Jennifer Pan, Satvik Sharma, and Chelsea Finn. MemER: Scaling up memory for robot control via experience retrieval. In *International Conference on Learning Representations*, 2026.
- [33] Richard S. Sutton, Doina Precup, and Satinder Singh. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 1999.
- [34] Chao Tang, Dehao Huang, Wenlong Dong, Ruinian Xu, and Hong Zhang. Foundationgrasp: Generalizable task-oriented grasping with foundation models. *IEEE Transactions on Automation Science and Engineering*, 2025.
- [35] Marcel Torne, Karl Pertsch, Homer Walke, Kyle Vedder, Suraj Nair, Brian Ichter, Allen Z Ren, Haohuan Wang, Jiaming Tang, Kyle Stachowicz, et al. MEM: Multi-scale embodied memory for vision language action models. *arXiv preprint arXiv:2603.03596*, 2026.
- [36] Fangyuan Wang, Ziyuan Wang, Guorui Pei, Mengshi Zhang, Canxi Liang, Jun Hu, Zhongxuan Li, Jinsong Wu, Ning Han, Zeqing Zhang, et al. World models for robotic manipulation: A survey. *arXiv preprint arXiv:2606.00113*, 2026.
- [37] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- [38] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, 2024.

- [39] Yue Yang, Linfeng Zhao, Mingyu Ding, Gedas Bertasius, and Daniel Szafrir. Boss: Benchmark for observation space shift in long-horizon task. *IEEE Robotics and Automation Letters*, 2025.
- [40] Yixian Zhang, Huanming Zhang, Feng Gao, Xiao Li, Zhihao Liu, Chunyang Zhu, Jiaying Qiu, Yuchen Yan, Jiyuan Liu, Wenhao Tang, et al. Harness vla: Steering frozen vlas into reliable manipulation primitives via memory-guided agents. *arXiv preprint arXiv:2607.08448*, 2026.
- [41] Meng Zheng, Samhita Marri, Anwesa Choudhuri, Benjamin Planche, Zhongpai Gao, Van Nguyen Nguyen, Terrence Chen, Girish Chowdhary, and Ziyang Wu. Failing forward: Adaptive failure-informed learning for vision-language-action models. *arXiv preprint arXiv:2605.08434*, 2026.
- [42] Xueyang Zhou, Yangming Xu, Guiyao Tie, Yongchao Chen, Guowen Zhang, Duanfeng Chu, Pan Zhou, and Lichao Sun. LIBERO-PRO: Towards robust and fair evaluation of vision-language-action models beyond memorization. *arXiv preprint arXiv:2510.03827*, 2025.